Build a Multi-Tool AI Agent using LangChain

Objective
Build an intelligent AI Agent using LangChain that automatically selects the correct tool based on the user query from a single chat interface.

Learning Objectives
Understand LangChain Agents, Tool Calling, Function Calling, Prompt Engineering, Agent Executor, Error Handling, Memory, and integration of multiple tools.

Functional Requirements
Implement Calculator, Wikipedia Search, Weather API, Web Search, and Local Document Search (RAG). The agent must automatically choose the correct tool.

Tools
1. Calculator: perform arithmetic and percentage calculations.
2. Wikipedia: retrieve and summarize factual information.
3. Weather: fetch current weather using any public API.
4. Web Search: search recent information using Tavily, DuckDuckGo or SerpAPI.
5. RAG: answer questions from uploaded PDFs using embeddings and FAISS/Chroma.

Memory
Maintain conversation history so follow-up questions use previous context.

Error Handling
Gracefully handle API failures, invalid input, missing PDFs, and timeouts.

Suggested Tech Stack
Python 3.11+, LangChain, OpenAI/Gemini, Qdrant, Streamlit/CLI.

Folder Structure
```
multi-tool-agent/
├── app.py
├── .env
├── requirements.txt
├── README.md
├── agents/
├── tools/
├── vectorstore/
├── documents/
└── utils/
```

Deliverables
GitHub repository, source code, README, .env.example, sample PDF, demo video/screenshots, assumptions.

Bonus

Implement with LangGraph, streaming responses, execution trace, persistent memory, parallel tool execution, and source citations.